

where.at.today

Master Project Guide for the Development Team - V3.7

A city-first event discovery platform on AWS.

Project	where.at.today
Prepared for	Product, engineering, design, and content operations teams
Version intent	3.7
Status	Reviewed

1. Product Vision and Core Experience

where.at.today should be built as a time-first, city-based event discovery platform. The product should answer one immediate question for the user: what is happening right now, tonight, this weekend, or near me?

The platform is not just a list of events. It is a city discovery engine. Users should be able to browse quickly, visually, and with as little friction as possible.

The experience should feel closer to a curated discovery feed than to a search-heavy ticketing website. That means the homepage, city pages, category pages, venue pages, and map flows should all be generated from strong data but presented in a simple editorial way.

1.1 Core principles

- Time-first discovery. Emphasize now, today, tonight, tomorrow, weekend, and this week.
- City-first navigation. Organize pages primarily around city context, such as /mumbai or /delhi.
- Event-instance-driven logic. Time-sensitive features should operate on actual occurrences, not generic event concepts.
- SEO by design. Pages, URLs, metadata, and structured data should be planned from day one.
- Venue-led data quality. Venues are the fastest path to reliable city coverage and location consistency.
- Manual curation first. Launch quality matters more than automation in the beginning.
- Visual clarity as product infrastructure. Category visuals, marker logic, and venue amenities must live in structured data, not in frontend hardcoding.

2. Recommended Technology Stack

Layer	Recommendation	Why
Frontend	Next.js + Tailwind CSS	Fast pages, strong SEO support, server-side rendering, and reusable UI components.
Backend	AWS Lambda + API Gateway to start	Low operations overhead and easy scaling for early traffic.
Database	PostgreSQL on AWS RDS	Strong fit for relational event data, joins, filters, and future PostGIS support.
Hosting	AWS Amplify	Simple deployment, SSL, CDN, and CI/CD.
Search	Algolia later	Useful when event volume, typo tolerance, and faceted search become more important.
Maps	Google Maps or Mapbox	Supports venue pins, near-me, and map discovery.
Email	Google Workspace	Operational mail such as hello@where.at.today and internal communication.

3. Logical System Architecture

At a high level, the platform should work like this:

Venue sources / organizers / manual research -> curation workflow -> PostgreSQL event catalog -> API layer / server-side queries -> Next.js pages and feeds -> SEO metadata and schema.org Event output

This split keeps the system clean. The content operations side focuses on adding and maintaining structured event data. The application layer focuses on ranking, filtering, visual presentation, and consistency across pages and maps.

4. Data Model Overview

Use UUIDs as the primary keys for all major entities. Slugs should be used for human-readable URLs. UUIDs are the stable internal identity. Slugs are the readable external route.

4.1 Why UUIDs are the right choice here

- They are safe to expose in APIs and integrations if needed.
- They avoid sequential ID leakage.
- They work well if the platform later splits into multiple services or ingestion jobs.
- They make imports and merges from multiple sources simpler.

4.2 Recommended table catalog

Table	What it stores	Why it exists separately	Maturity
cities	Supported city list, slug, timezone	Needed for city-first routing and consistent city metadata	Core
venues	Place where events happen	Many events reuse the same venue; location should not be repeated everywhere	Core
venue_services	Reusable venue amenities and services	Supports Airbnb-style venue experience display and venue filters	Core
venue_service_mappings	Links venue to services	One venue can expose many amenities; the service set must stay reusable	Core
organizers	Entity running the event	Useful for grouping, attribution, and future organizer pages	Core
event_series	Recurring or franchise-level event concept	Lets the team manage repeated events cleanly over time	Core
events	Canonical event shell / SEO object	Stores the main event identity and descriptive content	Core

event_instances	Actual occurrence with date/time and venue	All time-based logic depends on the real occurrence	Core
categories	Reusable primary categories plus visual metadata	Keeps category labels, icons, colors, and map markers clean and stable	Core
event_categories	Link between events and categories	One event can belong to multiple categories, but one must be primary	Core
tags	Reusable secondary descriptors	Adds flexible filtering without overloading categories	Core
event_tags	Link between events and tags	One event can have many tags	Core
event_media	Images or videos for an event	One event may have multiple media assets	Core
performers / event_performers	Artists, speakers, comedians, hosts	Supports performer-led discovery and richer event pages	Optional
ticket_sellers / event_ticket_offers	Providers, links, pricing, availability	Pricing and links can change per occurrence and seller	Core
subscriptions	Follow by category, venue, performer, or city	Supports future email alerts and repeat usage	Optional
users / user_profiles	Account identity, login method, and profile metadata	Supports optional accounts without blocking anonymous browsing	Optional
user_preferences / follows / favorites	Saved city, alert settings, favorite events and venues, followed categories	Supports personalization, retention, and faster repeat discovery	Optional
review workflow / audit_log	Maker-checker status, reviewer notes, and change history	Protects data integrity and creates accountability as the curation team grows	Optional

4.3 Modeling rules

- Every event with more than one category must still resolve to exactly one primary category for icon choice, map marker choice, default badge display, and canonical category context.
- Categories must own their visual identity through structured fields such as icon_key, map_marker_key, and color_hex.
- Venue services are attached to venues, not duplicated on events.
- Canonical event occurrence URLs must be unambiguous even when the same event has more than one instance on the same date.

- Published content changes to critical fields should pass through a maker-checker workflow before going live again.
- Optional user accounts should be modeled separately from public browsing so the platform remains low-friction and SEO-friendly.

5. Plain-English Explanation of the Core Tables

cities

This table stores the cities supported by the product, such as Mumbai, Delhi, and Bangalore. It is separate so the team can keep city names, slugs, and timezones consistent. A city page like /mumbai is powered by this table.

venues

This table stores the place where an event happens. A venue is a reusable entity such as Prithvi Theatre or antiSOCIAL. Because many events may happen at the same venue, the venue should be stored once and reused. This table should also hold the venue address and coordinates.

venue_services

This table stores reusable venue amenities such as Parking Available, Wheelchair Accessible, Air Conditioned, Outdoor Venue, or Bar Available. It exists so venue amenities can be displayed consistently and used in filters.

venue_service_mappings

This table links a venue to its services. It exists because one venue can expose many amenities and the same service can apply to many venues.

organizers

This table stores the organization, promoter, or venue team responsible for the event. It exists separately because one organizer can run many events and should not be duplicated in each event row.

event_series

This table stores the long-running concept behind repeated events. For example, a weekly open mic or a monthly market can be one series. This helps the team manage recurring events without rewriting everything each time.

events

This table stores the main event identity. Think of it as the publishable event shell that contains the title, slug, description, organizer link, editorial flags, and other stable details.

event_instances

This table stores the real-world occurrence of an event, with its own date, time, and venue. Features like Happening Now, This Weekend, Near Me, and map pins should all work from event_instances because users care about the actual occurrence they can attend.

categories

This table stores the main reusable buckets like Comedy, Music, Theatre, Workshops, or Food and Drink. In V3.7 it also stores the visual identity for those buckets so the app can render chips, cards, legends, and map markers consistently.

event_categories

This table links events to categories. It exists because one event can belong to more than one category. V3.7 keeps is_primary so the product always knows which category leads the visual presentation.

tags

This table stores secondary descriptors such as Free, Outdoor, Family Friendly, 18+, Open Air, Beginner Friendly, or Pet Friendly. Tags are different from categories. Categories describe what the event is. Tags describe extra attributes.

event_tags

This table links events to tags. It exists because one event can have many tags and the same tag can apply to many events.

event_media

This table stores media assets like hero images, posters, gallery images, or videos. An event may have multiple images, so storing them separately in event_media is a good design choice.

ticket_sellers and event_ticket_offers

These tables store ticket provider information and the actual seller offer for a specific occurrence. One event instance may have multiple sellers, and each seller may have its own pricing or link.

performers and event_performers

These tables become useful when the platform wants performer pages and richer event pages. A performer can appear in many events. An event can include many performers.

subscriptions

This table is optional for the first release but strongly recommended later. It stores follow and alert preferences for categories, venues, performers, or city-level interests. It is not the same as newsletter_subscribers: newsletter_subscribers owns email identity, consent, and unsubscribe state, while subscriptions owns topic-level follows and alert preferences.

For safer preference modeling, subscriptions should use reference_type + reference_id rather than a naked polymorphic UUID. That keeps the follow target explicit even when one subscription points to a category, venue, performer, city, or digest rule.

users, user_profiles, user_preferences, and user_follows / favorites

These tables are optional for launch, but they are the cleanest way to support account creation later without forcing login on every visitor. The users table should hold the core login identity. The user_profiles table should hold display details such as name, avatar, home city, and language. Preference and follow tables should hold saved city, followed categories, favorite venues, favorite events, and alert choices. This separation keeps authentication, profile data, and personalization concerns clean.

5.1 Source-of-truth matrix

The matrix below makes source-of-truth ownership explicit so product, design, content operations, and engineering all resolve the same questions to the same tables.

Concern	Source of truth	Notes
Event title, description, language, editorial flags	events	The event shell owns human-readable content and broad discovery metadata.
Occurrence time, venue mapping, lifecycle status, canonical URL	event_instances	Actual real-world occurrences drive time-based browsing, SEO pages, and lifecycle state.
Hero image and approved gallery assets	event_media	Approved media should be attached to the event shell, with exactly one approved hero image active at a time.
Ticket price and ticket availability	event_ticket_offers	Offer data can vary by instance and seller and should not be copied onto events.
Category visuals such as icon and marker color	categories	Visual identity belongs to taxonomy, not frontend hardcoding.
Venue amenities	venue_services + venue_service_mappings	Amenities are reusable venue attributes and should not be repeated per event.
Saved attendance intent and reminder state	user_saved_event_instances	Saving a specific occurrence is different from favoriting the broader event shell.

6. Core SQL Schema (Launch Version)

The following SQL is not every table the platform may ever need, but it is a strong launch-ready core that matches the product and page logic discussed so far. V3.7 keeps the full schema in one place, labels executable versus illustrative SQL more carefully, and hardens the final implementation rules.

Launch DDL - Core schema (Part 1 of 3)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

```
CREATE TABLE cities (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(100) NOT NULL,  
  slug VARCHAR(100) NOT NULL UNIQUE,  
  country VARCHAR(100) NOT NULL DEFAULT 'India',  
  timezone VARCHAR(100) NOT NULL DEFAULT 'Asia/Kolkata',  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE venues (  

```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
city_id UUID NOT NULL REFERENCES cities(id),
name VARCHAR(255) NOT NULL,
slug VARCHAR(255) NOT NULL UNIQUE,
address TEXT,
locality VARCHAR(100),
latitude NUMERIC(9,6),
longitude NUMERIC(9,6),
website_url TEXT,
description TEXT,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE venue_services (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(100) NOT NULL UNIQUE,
slug VARCHAR(100) NOT NULL UNIQUE,
icon_key VARCHAR(100),
group_name VARCHAR(50),
sort_order INTEGER NOT NULL DEFAULT 0,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE venue_service_mappings (
venue_id UUID NOT NULL REFERENCES venues(id),
venue_service_id UUID NOT NULL REFERENCES venue_services(id),
is_available BOOLEAN NOT NULL DEFAULT TRUE,
is_highlighted BOOLEAN NOT NULL DEFAULT FALSE,
PRIMARY KEY (venue_id, venue_service_id)
);
```

```
CREATE TABLE organizers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(255) NOT NULL,
slug VARCHAR(255) UNIQUE,
website_url TEXT,
instagram_url TEXT,
description TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```



```

CREATE TABLE event_series (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organizer_id UUID REFERENCES organizers(id),
  city_id UUID REFERENCES cities(id),
  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE,
  description TEXT,
  series_type VARCHAR(50),
  status VARCHAR(30) NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  series_id UUID REFERENCES event_series(id),
  organizer_id UUID REFERENCES organizers(id),
  city_id UUID NOT NULL REFERENCES cities(id),
  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL UNIQUE,
  short_description TEXT,
  full_description TEXT,
  language VARCHAR(50),
  min_age INTEGER,
  is_featured BOOLEAN NOT NULL DEFAULT FALSE,
  editor_pick BOOLEAN NOT NULL DEFAULT FALSE,
  status VARCHAR(30) NOT NULL DEFAULT 'draft',
  review_status VARCHAR(30) NOT NULL DEFAULT 'draft',
  reviewed_by UUID,
  reviewed_at TIMESTAMPTZ,
  review_notes TEXT,
  source_url TEXT,
  source_name VARCHAR(255),
  ingested_by UUID,
  verified_by UUID,
  last_verified_at TIMESTAMPTZ,
  content_confidence NUMERIC(3,2),
  image_verified BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

Launch DDL - Core schema (Part 2 of 3)

```

CREATE TABLE event_instances (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
event_id UUID NOT NULL REFERENCES events(id),
venue_id UUID NOT NULL REFERENCES venues(id),
start_at TIMESTAMPTZ NOT NULL,
end_at TIMESTAMPTZ,
doors_open_at TIMESTAMPTZ,
timezone VARCHAR(100) NOT NULL DEFAULT 'Asia/Kolkata',
status VARCHAR(30) NOT NULL DEFAULT 'scheduled',
review_status VARCHAR(30) NOT NULL DEFAULT 'draft',
reviewed_by UUID,
reviewed_at TIMESTAMPTZ,
review_notes TEXT,
canonical_url TEXT UNIQUE,
inventory_status VARCHAR(30) NOT NULL DEFAULT 'available',
source_url TEXT,
source_name VARCHAR(255),
ingested_by UUID,
verified_by UUID,
last_verified_at TIMESTAMPTZ,
content_confidence NUMERIC(3,2),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE categories (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name VARCHAR(100) NOT NULL UNIQUE,
slug VARCHAR(100) NOT NULL UNIQUE,
icon_key VARCHAR(100),
map_marker_key VARCHAR(100),
color_hex VARCHAR(10),
priority INTEGER NOT NULL DEFAULT 0,
sort_order INTEGER NOT NULL DEFAULT 0,
is_discoverable BOOLEAN NOT NULL DEFAULT TRUE,
is_filterable BOOLEAN NOT NULL DEFAULT TRUE,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE event_categories (
event_id UUID NOT NULL REFERENCES events(id),
category_id UUID NOT NULL REFERENCES categories(id),
is_primary BOOLEAN NOT NULL DEFAULT FALSE,
PRIMARY KEY (event_id, category_id)
);

```

```

);

CREATE TABLE tags (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL UNIQUE,
  slug VARCHAR(100) NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE event_tags (
  event_id UUID NOT NULL REFERENCES events(id),
  tag_id UUID NOT NULL REFERENCES tags(id),
  PRIMARY KEY (event_id, tag_id)
);

CREATE TABLE event_media (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID NOT NULL REFERENCES events(id),
  media_type VARCHAR(30) NOT NULL,
  media_url TEXT NOT NULL,
  alt_text VARCHAR(255),
  aspect_ratio VARCHAR(20),
  is_hero BOOLEAN NOT NULL DEFAULT FALSE,
  is_approved BOOLEAN NOT NULL DEFAULT FALSE,
  approval_status VARCHAR(30) NOT NULL DEFAULT 'draft',
  reviewed_by UUID,
  reviewed_at TIMESTAMPTZ,
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

Launch DDL - Core schema (Part 3 of 3)

```

CREATE TABLE performers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE,
  bio TEXT,
  image_url TEXT,
  website_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE event_performers (
  event_id UUID NOT NULL REFERENCES events(id),

```

```

performer_id UUID NOT NULL REFERENCES performers(id),
role VARCHAR(100),
PRIMARY KEY (event_id, performer_id)
);

CREATE TABLE ticket_sellers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE,
  website_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE event_ticket_offers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_instance_id UUID NOT NULL REFERENCES event_instances(id),
  seller_id UUID NOT NULL REFERENCES ticket_sellers(id),
  seller_event_url TEXT NOT NULL,
  price_min NUMERIC(10,2),
  price_max NUMERIC(10,2),
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  availability_status VARCHAR(30) NOT NULL DEFAULT 'available',
  last_synced_at TIMESTAMPTZ,
  review_status VARCHAR(30) NOT NULL DEFAULT 'draft',
  reviewed_by UUID,
  reviewed_at TIMESTAMPTZ,
  review_notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- subscriptions are defined in Section 6.2 after users and newsletter_subscribers so ownership can
-- reference either account identities or newsletter-only identities safely.

```

6.1 Index and constraint placement note

The core launch index and constraint DDL appears later in this section as the single authoritative executable block for the core launch schema. Optional user-account and review-table constraints appear in separate Optional DDL blocks near the optional tables they depend on. Governance and workflow sections may reference these rules conceptually, but executable DDL should live in one authoritative place per schema tier.

```

Core Launch DDL - Recommended indexes and constraints
CREATE INDEX idx_venues_city_id ON venues(city_id);

```

```

CREATE INDEX idx_event_instances_event_id ON event_instances(event_id);
CREATE INDEX idx_event_instances_venue_id ON event_instances(venue_id);
CREATE INDEX idx_event_instances_start_at ON event_instances(start_at);
CREATE INDEX idx_event_instances_end_at ON event_instances(end_at);
CREATE INDEX idx_event_instances_status ON event_instances(status);
CREATE INDEX idx_events_city_id ON events(city_id);
CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_events_review_status ON events(review_status);
CREATE INDEX idx_categories_slug ON categories(slug);
CREATE INDEX idx_venue_service_mappings_venue_id ON venue_service_mappings(venue_id);

CREATE UNIQUE INDEX uniq_primary_category_per_event
  ON event_categories(event_id)
  WHERE is_primary = TRUE;

ALTER TABLE events
  ADD CONSTRAINT chk_events_status
  CHECK (status IN ('draft', 'published', 'archived'));

ALTER TABLE events
  ADD CONSTRAINT chk_events_review_status
  CHECK (review_status IN ('draft', 'pending_review', 'changes_requested', 'approved', 'rejected'));

ALTER TABLE event_instances
  ADD CONSTRAINT chk_event_instances_status
  CHECK (status IN ('scheduled', 'postponed', 'cancelled', 'completed'));

ALTER TABLE event_instances
  ADD CONSTRAINT chk_event_instances_inventory_status
  CHECK (inventory_status IN ('available', 'low', 'sold_out', 'unavailable'));

ALTER TABLE event_media
  ADD CONSTRAINT chk_event_media_approval_status
  CHECK (approval_status IN ('draft', 'pending_review', 'changes_requested', 'approved', 'rejected'));

ALTER TABLE event_ticket_offers
  ADD CONSTRAINT chk_event_ticket_offers_review_status
  CHECK (review_status IN ('draft', 'pending_review', 'changes_requested', 'approved', 'rejected'));

ALTER TABLE event_ticket_offers
  ADD CONSTRAINT chk_event_ticket_offers_availability_status
  CHECK (availability_status IN ('available', 'sold_out', 'waitlist', 'unknown'));

CREATE UNIQUE INDEX ux_event_media_one_approved_hero

```

```
ON event_media(event_id)
WHERE is_hero = TRUE AND is_approved = TRUE;
```

6.2 Optional user account and profile schema

The platform should remain fully browseable without login. However, the following optional schema gives the product a clean path to account creation, saved preferences, favorites, and follows when the team is ready.

Optional DDL - User account and profile schema

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  phone VARCHAR(30),
  auth_provider VARCHAR(30) NOT NULL DEFAULT 'email',
  password_hash TEXT,
  status VARCHAR(30) NOT NULL DEFAULT 'active',
  email_verified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (email = lower(btrim(email))),
  CHECK (auth_provider IN ('email', 'google', 'apple')),
  CHECK (status IN ('active', 'suspended', 'deleted'))
);
```

Optional DDL - User account and profile schema

```
CREATE TABLE user_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id),
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  display_name VARCHAR(150),
  avatar_url TEXT,
  city_id UUID REFERENCES cities(id),
  preferred_language VARCHAR(50),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Optional DDL - User account and profile schema

```
CREATE TABLE user_preferences (
  user_id UUID PRIMARY KEY REFERENCES users(id),
  preferred_city_id UUID REFERENCES cities(id),
  radius_km INTEGER,
  wants_nearby_alerts BOOLEAN NOT NULL DEFAULT FALSE,
  wants_weekend_alerts BOOLEAN NOT NULL DEFAULT FALSE,
  wants_editor_picks BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Optional DDL - User account and profile schema

```
CREATE TABLE user_favorite_events (
  user_id UUID NOT NULL REFERENCES users(id),
  event_id UUID NOT NULL REFERENCES events(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, event_id)
);
```

```
CREATE TABLE user_saved_event_instances (
  user_id UUID NOT NULL REFERENCES users(id),
  event_instance_id UUID NOT NULL REFERENCES event_instances(id),
  reminder_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```

PRIMARY KEY (user_id, event_instance_id)
);

CREATE TABLE user_favorite_venues (
  user_id UUID NOT NULL REFERENCES users(id),
  venue_id UUID NOT NULL REFERENCES venues(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, venue_id)
);

CREATE TABLE user_follow_categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  category_id UUID NOT NULL REFERENCES categories(id),
  city_id UUID REFERENCES cities(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
);

CREATE UNIQUE INDEX ux_user_follow_categories_global
ON user_follow_categories (user_id, category_id)
WHERE city_id IS NULL;

CREATE UNIQUE INDEX ux_user_follow_categories_city
ON user_follow_categories (user_id, category_id, city_id)
WHERE city_id IS NOT NULL;

```

user_follow_categories supports both global follows and city-scoped follows. Use city_id = NULL for a global category follow such as 'Comedy everywhere'. Use a populated city_id for a city-scoped follow such as 'Comedy in Mumbai'. The partial unique indexes above prevent duplicates in both cases.

6.2.1 Internal staff actor identity

All *_by fields used for review, ingestion, verification, and audit should represent internal operations staff identities, not public visitor accounts. At launch, core content tables may keep these columns as raw UUID values so the platform can integrate with any admin auth provider, but the semantic contract should remain staff_users.id or an equivalent internal actor subject.

Optional DDL - Internal staff identity for operations and review

```

CREATE TABLE staff_users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  display_name VARCHAR(150),
  role_key VARCHAR(30) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (role_key IN ('maker', 'checker', 'admin')),
  CHECK (email = lower(btrim(email)))
);

```

6.2.2 Newsletter-only subscription and later account upgrade

The product should support low-friction newsletter subscription without requiring a full account. Email must be stored normalized as lower(trim(email)) across users, newsletter_subscribers, and staff_users. Account signup should first attempt to match an existing verified newsletter_subscribers row by normalized email before deciding whether a new newsletter identity is needed.

Optional DDL - Newsletter-only subscription

```

CREATE TABLE newsletter_subscribers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,

```

```

status VARCHAR(30) NOT NULL DEFAULT 'pending_verification',
source VARCHAR(50) NOT NULL DEFAULT 'website',
consent_text_version VARCHAR(50),
user_id UUID REFERENCES users(id),
UNIQUE (user_id),
subscribed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
verified_at TIMESTAMPTZ,
unsubscribed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CHECK (status IN ('pending_verification', 'active', 'unsubscribed', 'bounced')),
CHECK (source IN ('website', 'homepage', 'footer', 'event_page', 'signup', 'import', 'admin')),
CHECK (email = lower(btrim(email)))
);

```

Optional DDL - Topic subscriptions and alert preferences

```

CREATE TABLE subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  newsletter_subscriber_id UUID REFERENCES newsletter_subscribers(id),
  user_id UUID REFERENCES users(id),
  subscription_type VARCHAR(30) NOT NULL,
  reference_type VARCHAR(30),
  reference_id UUID,
  city_id UUID REFERENCES cities(id),
  status VARCHAR(30) NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CHECK (
    newsletter_subscriber_id IS NOT NULL
    OR user_id IS NOT NULL
  ),
  CHECK (status IN ('active', 'paused', 'unsubscribed')),
  CHECK (subscription_type IN ('newsletter', 'follow', 'alert')),
  CHECK (reference_type IS NULL OR reference_type IN ('city', 'category', 'venue', 'performer')),
  CHECK (
    (subscription_type = 'newsletter' AND reference_type IS NULL AND reference_id IS NULL)
    OR
    (subscription_type IN ('follow', 'alert') AND reference_type IS NOT NULL AND reference_id IS NOT
    NULL)
  )
);

```

```

CREATE UNIQUE INDEX ux_subscriptions_user_active_target ON subscriptions (
  user_id,
  subscription_type,
  reference_type,
  COALESCE(reference_id, '00000000-0000-0000-0000-000000000000'::uuid),
  COALESCE(city_id, '00000000-0000-0000-0000-000000000000'::uuid)
)
WHERE user_id IS NOT NULL AND status = 'active';

```

```

CREATE UNIQUE INDEX ux_subscriptions_newsletter_active_target ON subscriptions (
  newsletter_subscriber_id,
  subscription_type,
  reference_type,
  COALESCE(reference_id, '00000000-0000-0000-0000-000000000000'::uuid),
  COALESCE(city_id, '00000000-0000-0000-0000-000000000000'::uuid)
)
WHERE newsletter_subscriber_id IS NOT NULL AND status = 'active';

```

Supported identity and consent scenarios: (1) visitor subscribes first, no account yet - create only newsletter_subscribers plus any linked subscriptions; (2) visitor subscribes first, then creates an account later - create users, link users.id into newsletter_subscribers.user_id for the matching verified normalized

email, preserve consent history, backfill subscriptions.user_id, and update the existing subscription rows rather than creating duplicate parallel rows for the same target while keeping newsletter_subscriber_id as the consent lineage anchor; (3) visitor creates an account but does not subscribe - create only users. Creating an account must never auto-subscribe a person to newsletter. Newsletter without account, account without newsletter, account plus newsletter, and newsletter first then account later are all valid. The linking of newsletter_subscribers, creation of users, and backfill/update of related subscriptions should execute in a single transaction so identity linkage and preference ownership cannot partially succeed. For subscriptions, city_id is optional. It must be NULL for global newsletter subscriptions, may be NULL for global follows or alerts, and should be populated only when the follow or alert is intentionally city-scoped.

6.2.3 Draft autosave, version snapshots, and review diff support

Drafts should not live only in one giant JSON blob. The main relational tables remain the canonical source of truth for events, instances, media, offers, venues, and taxonomy. JSONB is still useful for autosave snapshots, review diffs, and version history because forms can span multiple entities before they are valid enough for structured commit. The recommended pattern is: autosave draft snapshots while a maker is typing, write structured rows on explicit save, and generate version plus diff records on submit-for-review.

Optional DDL - Draft autosave and recovery

```
CREATE TABLE draft_snapshots (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_type VARCHAR(50) NOT NULL,  
  entity_id UUID,  
  maker_user_id UUID NOT NULL REFERENCES staff_users(id),  
  draft_payload JSONB NOT NULL,  
  autosaved_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Optional DDL - Version snapshots

```
CREATE TABLE entity_versions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_type VARCHAR(50) NOT NULL,  
  entity_id UUID NOT NULL,  
  version_number INTEGER NOT NULL,  
  snapshot JSONB NOT NULL,  
  created_by UUID REFERENCES staff_users(id),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE (entity_type, entity_id, version_number)  
);
```

Optional DDL - Review queue with before/after compare support

```
CREATE TABLE entity_reviews (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_type VARCHAR(50) NOT NULL,  
  entity_id UUID NOT NULL,  
  version_from INTEGER,  
  version_to INTEGER,  
  before_payload JSONB,  
  after_payload JSONB NOT NULL,  
  changed_fields JSONB,  
  review_status VARCHAR(30) NOT NULL DEFAULT 'pending_review',  
  review_notes TEXT,  
  submitted_by UUID REFERENCES staff_users(id),  
  reviewed_by UUID REFERENCES staff_users(id),  
  submitted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  reviewed_at TIMESTAMPTZ,
```

```
CHECK (review_status IN ('draft', 'pending_review', 'changes_requested', 'approved', 'rejected'))
);
```

Optional DDL - User/review extension indexes and constraints

Use the inline CHECK constraints defined in the optional CREATE TABLE statements as the executable source of truth for users and entity_reviews. Do not repeat duplicate ALTER TABLE ADD CONSTRAINT statements for those same rules in extension installs.

6.3 Migration notes for upgrading older schemas and optional operational extensions

Greenfield V3.7 builds should use the launch DDL above as the source of truth. This section groups legacy-upgrade guidance alongside a small optional operational extension so teams can see both without confusing them with the core launch DDL.

Migration Example - legacy upgrades only

If you are upgrading from a pre-V3.7 schema, add missing review, provenance, canonical occurrence, and media-governance columns only where they do not already exist. Do not run historical ALTER TABLE snippets against the full V3.7 launch schema unchanged.

Optional DDL - Operational audit log

```
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entity_type VARCHAR(50) NOT NULL,
  entity_id UUID NOT NULL,
  action_type VARCHAR(50) NOT NULL,
  performed_by UUID,
  action_notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

6.4 Status vocabularies and allowed enums

The platform should define explicit status vocabularies so admin, API, analytics, and automation use the same state language. The values below are recommended launch defaults.

- events.status: draft | published | archived
- events.review_status: draft | pending_review | changes_requested | approved | rejected
- event_instances.status: scheduled | postponed | cancelled | completed
- event_instances.inventory_status: available | low | sold_out | unavailable
- event_ticket_offers.availability_status: available | sold_out | waitlist | unknown
- users.status: active | suspended | deleted

Recommended simplification: use lifecycle/status columns only for whether a record is live and visible, and use review_status columns for approval workflow. For example, events.status should answer whether the event shell is draft, published, or archived, while events.review_status should answer whether the current draft is waiting for approval.

- event_media.approval_status: draft | pending_review | changes_requested | approved | rejected
- event_ticket_offers.review_status: draft | pending_review | changes_requested | approved | rejected
- entity_reviews.review_status: draft | pending_review | changes_requested | approved | rejected

Suggested transition rules: lifecycle state and review state should remain separate. Typical launch behavior is events.status moving draft -> published -> archived, while events.review_status moves draft -> pending_review -> changes_requested -> approved -> rejected as needed before publication. The shared entity_reviews table should stay leaner because it is a review artifact rather than a live publication surface.

6.4.1 Recommended DB enforcement pattern

Illustrative note - Status enforcement should be implemented in the single authoritative launch DDL/index and constraint subsection later in Section 6. Do not duplicate executable CHECK-constraint DDL in multiple places. This section defines the recommended state model conceptually; the launch schema block is the executable source of truth.

6.5 Auditability and provenance fields

Manual curation quality improves when the system records where content came from, who entered it, and when it was last verified. In V3.7 the main launch DDL carries the fuller provenance set on events and event_instances. event_media and event_ticket_offers carry approval/review fields in launch DDL, and teams that want the same fuller provenance footprint there can add it through the optional migration pattern below.

Migration Example - provenance backfill for older schemas only

If an older schema does not yet track source_url, source_name, ingested_by, verified_by, last_verified_at, content_confidence, or image_verified, backfill those columns through a targeted migration rather than by editing the launch DDL in place.

Actor identity rule: created_by, updated_by, submitted_by, reviewed_by, ingested_by, verified_by, and performed_by should all represent internal staff/admin identities. They should not point to public browsing users. If the team does not yet introduce a dedicated staff_users table in the first deployment, it should still keep one stable internal actor-ID contract across admin, review, and audit systems.

Recommended semantics: source_url is the original listing or calendar page; source_name is the human-readable source label; ingested_by is the maker; verified_by is the checker; last_verified_at is the last manual confirmation timestamp; content_confidence is a simple quality score such as 0.60, 0.80, or 1.00.

6.6 Media governance rules

event_media should follow explicit publishing rules so content operations and frontend rendering stay consistent.

- Exactly one hero image per event should be approved at any time.
- Additional gallery images may exist, but they should not replace the hero image unless explicitly promoted.
- If both poster art and landscape art exist, landscape should be preferred for homepage cards and Open Graph images, while poster art may remain available inside galleries.
- If no valid approved hero exists, the event should fail the publish-ready gate unless the product team has defined a safe fallback placeholder.
- Broken image URLs, unsupported file types, or images below the minimum quality threshold should fail validation.

- If venue imagery is planned, it should live in a separate venue_media table instead of being mixed into event_media.

Optional DDL - Venue media schema

```
CREATE TABLE venue_media (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  venue_id UUID NOT NULL REFERENCES venues(id),  
  media_type VARCHAR(30) NOT NULL,  
  media_url TEXT NOT NULL,  
  alt_text VARCHAR(255),  
  is_approved BOOLEAN NOT NULL DEFAULT FALSE,  
  reviewed_by UUID,  
  reviewed_at TIMESTAMPTZ,  
  sort_order INTEGER NOT NULL DEFAULT 0,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Recommended launch media rules: require one approved hero image, prefer landscape-first assets for cards and sharing, preview images before save, and distinguish hero, poster, gallery, and fallback assets clearly in admin.

Illustrative note - The authoritative unique index for one approved hero image per event lives in the Core Launch DDL/index and constraint block later in Section 6. Do not duplicate that executable index in migration or governance sections.

This index makes the hero-image rule concrete: one event can have many images, but only one approved hero can be active at a time.

6.7 Timezone discipline

Time is the product axis, so timestamp handling must be explicit even if the launch city is Mumbai.

- Store canonical timestamps in UTC using TIMESTAMPTZ columns in the database, including optional user-account and audit tables unless a field is intentionally date-only or local-text metadata.
- Keep city.timezone and event_instances.timezone for display and conversion logic.
- Return API timestamps in ISO 8601 format with offset so clients never guess the intended local time.
- If a curator receives only a local clock time from an organizer, ingestion should convert that local time into UTC before save and preserve the source timezone for display.
- If the platform expands beyond India, daylight saving behavior should be resolved through IANA timezone rules rather than custom offsets.

7. Sample Data Example

The example below shows how one event flows across the schema.

Illustrative SQL - Sample data (Part 1 of 2)

```
INSERT INTO cities (id, name, slug) VALUES
```

```
('11111111-1111-1111-1111-111111111111', 'Mumbai', 'mumbai');
```

```
INSERT INTO venues (id, city_id, name, slug, address, locality, latitude, longitude) VALUES
```

```
('22222222-2222-2222-2222-222222222222', '11111111-1111-1111-1111-111111111111',  
'The Habitat', 'the-habitat-khar', '3rd Floor, Unicontinental, Khar West, Mumbai',
```

```
'Khar', 19.069600, 72.836500);
```

```
INSERT INTO venue_services (id, name, slug, icon_key, group_name, sort_order) VALUES  
(('12121212-1212-1212-1212-121212121212', 'Parking Available', 'parking-available', 'parking',  
'Access', 1),  
(('13131313-1313-1313-1313-131313131313', 'Air Conditioned', 'air-conditioned', 'ac', 'Comfort', 2);
```

```
INSERT INTO venue_service_mappings (venue_id, venue_service_id, is_available, is_highlighted) VALUES  
(('22222222-2222-2222-2222-222222222222', '12121212-1212-1212-1212-121212121212', TRUE,  
TRUE),  
(('22222222-2222-2222-2222-222222222222', '13131313-1313-1313-1313-131313131313', TRUE,  
TRUE);
```

```
INSERT INTO organizers (id, name, slug) VALUES  
(('33333333-3333-3333-3333-333333333333', 'Comedy Club India', 'comedy-club-india');
```

```
INSERT INTO event_series (id, organizer_id, city_id, title, slug, description) VALUES  
(('44444444-4444-4444-4444-444444444444', '33333333-3333-3333-3333-333333333333',  
'11111111-1111-1111-1111-111111111111', 'Open Mic Comedy Night',  
'open-mic-comedy-night', 'Weekly comedy open mic series in Mumbai.');
```

```
INSERT INTO events (id, series_id, organizer_id, city_id, title, slug, short_description,  
full_description, editor_pick, status, review_status, source_url, source_name, last_verified_at,  
content_confidence, image_verified) VALUES  
(('55555555-5555-5555-5555-555555555555', '44444444-4444-4444-4444-444444444444',  
'33333333-3333-3333-3333-333333333333', '11111111-1111-1111-1111-111111111111',  
'Open Mic Comedy Night', 'open-mic-comedy-night-habitat-mumbai',  
'Weekly stand-up comedy open mic in Khar',  
'A weekly open mic featuring upcoming comics and guest headliners',  
TRUE, 'published', 'approved', 'https://thehabitat.in/upcoming/open-mic-comedy-night',  
'The Habitat Listings', '2026-04-05T10:00:00+05:30', 0.95, TRUE);
```

Illustrative SQL - Sample data (Part 2 of 2)

```
INSERT INTO event_instances (id, event_id, venue_id, start_at, end_at, canonical_url, status,  
review_status, source_url, source_name, last_verified_at, content_confidence) VALUES  
(('66666666-6666-6666-6666-666666666666', '55555555-5555-5555-5555-555555555555',  
'22222222-2222-2222-2222-222222222222',  
'2026-04-10T20:30:00+05:30', '2026-04-10T22:30:00+05:30',  
'/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030', 'scheduled',  
'approved', 'https://thehabitat.in/upcoming/open-mic-comedy-night',  
'The Habitat Listings', '2026-04-05T10:00:00+05:30', 0.95);
```

```
INSERT INTO categories (id, name, slug, icon_key, map_marker_key, color_hex, priority) VALUES  
(('77777777-7777-7777-7777-777777777777', 'Comedy', 'comedy', 'mic', 'marker-comedy', '#FF6B6B',
```

```

10),
('78787878-7878-7878-787878787878', 'Nightlife', 'nightlife', 'moon', 'marker-nightlife',
'#5B63F6', 5);

INSERT INTO event_categories (event_id, category_id, is_primary) VALUES
('55555555-5555-5555-5555-555555555555', '77777777-7777-7777-7777-777777777777', TRUE),
('55555555-5555-5555-5555-555555555555', '78787878-7878-7878-7878-787878787878', FALSE);

INSERT INTO tags (id, name, slug) VALUES
('88888888-8888-8888-8888-888888888888', '18+', '18-plus');

INSERT INTO event_tags (event_id, tag_id) VALUES
('55555555-5555-5555-5555-555555555555', '88888888-8888-8888-8888-888888888888');

INSERT INTO event_media (id, event_id, media_type, media_url, alt_text, aspect_ratio,
is_hero, is_approved, approval_status, reviewed_at, sort_order) VALUES
('99999999-9999-9999-9999-999999999999', '55555555-5555-5555-5555-555555555555',
'hero', 'https://cdn.where.at.today/events/open-mic-hero.jpg',
'Open Mic Comedy Night hero image', '16:9', TRUE, TRUE, 'approved',
'2026-04-05T10:15:00+05:30', 1);

INSERT INTO ticket_sellers (id, name, slug, website_url) VALUES
('aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', 'BookMyShow', 'bookmyshow', 'https://bookmyshow.com');

INSERT INTO event_ticket_offers (id, event_instance_id, seller_id, seller_event_url,
price_min, price_max, currency, availability_status, review_status) VALUES
('bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbb', '66666666-6666-6666-6666-666666666666',
'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa', 'https://seller.example/open-mic-apr-10',
299.00, 499.00, 'INR', 'available', 'approved');

```

8. Page Types, Routing Logic, and Visual Discovery

8.1 Homepage

The homepage should immediately orient the user around a city and a time window. It should be a modular dashboard with sections such as Happening Now, Tonight, This Weekend, Editor Picks, Recently Added, Browse by Category, and Popular Venues.

8.2 City pages

A city page like /mumbai is one of the most important SEO and navigation pages. It should group content into live, today, tonight, weekend, and category-based blocks.

8.3 Category pages

Pages like /mumbai/comedy or /mumbai/music should show events for that city and category, ordered by time and optionally boosted by editorial ranking. Category pages should also define the visual language of discovery by inheriting the category icon, color, and map marker style.

8.4 Time pages

Pages like /mumbai/today, /mumbai/tonight, and /mumbai/weekend should match how users actually search for plans.

8.5 Event pages

Each real occurrence should have a dedicated event page for strong SEO and correct structured data. The public route should be occurrence-specific so it remains canonical and unambiguous.

8.6 Venue pages

A page like /venue/prithvi-theatre should show venue information, highlighted services, a full amenity list, upcoming events, map location, and past events.

8.7 Map view and visual discovery

- Each event_instance should appear as a pin in map view.
- The pin icon is driven by the event's primary category.
- The pin color comes from the category color_hex field.
- The selected venue or event popover should show highlighted venue services as icons.
- Map view should support category filtering, time filtering, radius filtering, Happening Now, and This Weekend.
- When density becomes visually noisy, frontend may cluster markers, but expanded markers should still preserve category identity.

8.8 Suggested homepage sections

Section	Purpose	Data source
Hero	Set city context and time filters	cities + user preference + quick filters
Happening Now	Show currently live events	event_instances with NOW between start_at and end_at
Tonight	Show strongest evening options	future instances later today
This Weekend	Help users plan	instances within weekend window
Editor Picks	Show curation and quality	events.editor_pick = true
Recently Added	Make the platform feel alive	events ordered by created_at desc
Browse by Category	Support discovery by interest	categories + event_categories + category visuals
Popular Venues	Give venue-level browsing paths	venues with active upcoming instances

9. Query Patterns the Team Will Need

9.1 Event page fetch query (upgraded and complete)

Earlier revisions restored venue services and primary category, but the event payload also needs instance, categories, tags, approved media, approved offers, and canonical occurrence context. V3.7 keeps the full event occurrence payload, uses event_instances.canonical_url as the public route source of truth, and limits live page data to approved assets and approved offers only. This query runs only after public route resolution has already identified event_instance_id from the canonical request path.

Illustrative Query - Event occurrence payload

```
SELECT json_build_object(  
  'id', e.id,  
  'slug', e.slug,  
  'title', e.title,  
  'shortDescription', e.short_description,  
  'fullDescription', e.full_description,  
  'editorPick', e.editor_pick,  
  'city', json_build_object(  
    'id', c.id,  
    'name', c.name,  
    'slug', c.slug  
  ),  
  'venue', json_build_object(  
    'id', v.id,  
    'name', v.name,  
    'slug', v.slug,  
    'address', v.address,  
    'locality', v.locality,  
    'latitude', v.latitude,  
    'longitude', v.longitude,  
    'services', (  
      SELECT COALESCE(json_agg(json_build_object(  
        'name', vs.name,  
        'slug', vs.slug,  
        'iconKey', vs.icon_key,  
        'highlighted', vsm.is_highlighted  
      ) ORDER BY vs.sort_order), '[]'::json)  
    FROM venue_service_mappings vsm  
    JOIN venue_services vs ON vs.id = vsm.venue_service_id  
    WHERE vsm.venue_id = v.id AND vsm.is_available = TRUE  
    )  
  ),  
  'instance', json_build_object(  
    'id', ei.id,  
    'canonicalUrl', ei.canonical_url,
```



```

'startAt', ei.start_at,
'endAt', ei.end_at,
'timezone', ei.timezone,
'status', ei.status,
'inventoryStatus', ei.inventory_status
),
'primaryCategory', (
  SELECT json_build_object(
    'name', cat.name,
    'slug', cat.slug,
    'iconKey', cat.icon_key,
    'mapMarkerKey', cat.map_marker_key,
    'colorHex', cat.color_hex
  )
  FROM event_categories ec
  JOIN categories cat ON cat.id = ec.category_id
  WHERE ec.event_id = e.id AND ec.is_primary = TRUE
  LIMIT 1
),
'categories', (
  SELECT COALESCE(json_agg(json_build_object(
    'name', cat.name,
    'slug', cat.slug
  ) ORDER BY cat.sort_order), '[]'::json)
  FROM event_categories ec
  JOIN categories cat ON cat.id = ec.category_id
  WHERE ec.event_id = e.id
),
'tags', (
  SELECT COALESCE(json_agg(json_build_object(
    'name', t.name,
    'slug', t.slug
  )), '[]'::json)
  FROM event_tags et
  JOIN tags t ON t.id = et.tag_id
  WHERE et.event_id = e.id
),
'media', (
  SELECT COALESCE(json_agg(json_build_object(
    'url', em.media_url,
    'type', em.media_type,
    'altText', em.alt_text
  ) ORDER BY CASE WHEN em.is_hero THEN 0 ELSE 1 END, em.sort_order), '[]'::json)
  FROM event_media em

```

```

WHERE em.event_id = e.id
AND em.is_approved = TRUE
AND em.approval_status = 'approved'
),
'offers', (
SELECT COALESCE(json_agg(json_build_object(
'sellerName', ts.name,
'url', eto.seller_event_url,
'priceMin', eto.price_min,
'priceMax', eto.price_max,
'currency', eto.currency,
'availabilityStatus', eto.availability_status
)), '[]'::json)
FROM event_ticket_offers eto
JOIN ticket_sellers ts ON ts.id = eto.seller_id
WHERE eto.event_instance_id = ei.id
AND eto.review_status = 'approved'
AND eto.availability_status IN ('available', 'sold_out', 'waitlist', 'unknown')
)
) AS event_page_payload
FROM events e
JOIN cities c ON c.id = e.city_id
JOIN event_instances ei ON ei.event_id = e.id
JOIN venues v ON v.id = ei.venue_id
WHERE e.slug = :event_slug
AND ei.id = :instance_id
AND e.status = 'published';

```

9.2 Nearby events query (map-ready, approved-offer aware, coordinate-guarded, hero-thumbnail aware, and null-safe when end_at is missing)

For production safety, compute distance_km in a CTE first and then filter in an outer WHERE clause. Clamp the cosine input with LEAST(1, GREATEST(-1, ...)) before acos(...) so floating-point drift cannot break the query. Map previews should continue to return inventoryStatus, a hero-first approved-media thumbnailUrl, and only approved ticket-offer pricing.

Illustrative Query - Hardened nearby/map preview pattern

```

WITH distance_base AS (
SELECT
  ei.id AS event_instance_id,
  e.slug,
  e.title,
  ei.start_at,
  ei.inventory_status,
  v.name AS venue_name,
  v.slug AS venue_slug,
  v.latitude,
  v.longitude,

```

```

6371 * acos(
    LEAST(1, GREATEST(-1,
        cos(radians(:user_lat)) * cos(radians(v.latitude)) *
        cos(radians(v.longitude) - radians(:user_lng)) +
        sin(radians(:user_lat)) * sin(radians(v.latitude))
    ))
) AS distance_km
FROM event_instances ei
JOIN events e ON e.id = ei.event_id
JOIN venues v ON v.id = ei.venue_id
WHERE e.status = 'published'
    AND ei.status = 'scheduled'
    AND COALESCE(ei.end_at, ei.start_at) >= NOW()
    AND v.latitude IS NOT NULL
    AND v.longitude IS NOT NULL
)
SELECT *
FROM distance_base
WHERE distance_km <= :radius_km
ORDER BY distance_km ASC, start_at ASC
LIMIT 50;

```

Illustrative Query - Map/event preview query

WITH preview_base AS (

SELECT

ei.id AS event_instance_id,

e.id AS event_id,

e.title,

e.slug,

e.editor_pick,

v.name AS venue_name,

v.slug AS venue_slug,

v.latitude,

v.longitude,

ei.start_at,

ei.end_at,

ei.inventory_status,

ei.canonical_url,

pc.slug AS primary_category_slug,

pc.name AS primary_category_name,

pc.icon_key AS primary_category_icon_key,

pc.map_marker_key AS primary_category_map_marker_key,

pc.color_hex AS primary_category_color_hex,

COALESCE((

SELECT MIN(eto.price_min)

FROM event_ticket_offers eto

WHERE eto.event_instance_id = ei.id

AND eto.review_status = 'approved'

AND eto.availability_status IN ('available', 'sold_out', 'waitlist', 'unknown')

), NULL) AS price_min,

COALESCE((

SELECT em.media_url

```

FROM event_media em
WHERE em.event_id = e.id
AND em.is_approved = TRUE
AND em.approval_status = 'approved'
ORDER BY CASE WHEN em.is_hero THEN 0 ELSE 1 END, em.sort_order ASC
LIMIT 1
), NULL) AS thumbnail_url,
COALESCE((
SELECT json_agg(json_build_object(
'name', vs.name,
'iconKey', vs.icon_key,
'highlighted', vsm.is_highlighted
) ORDER BY vs.sort_order)
FROM venue_service_mappings vsm
JOIN venue_services vs ON vs.id = vsm.venue_service_id
WHERE vsm.venue_id = v.id
AND vsm.is_available = TRUE
AND vsm.is_highlighted = TRUE
), '[]::json') AS highlighted_services,
CASE WHEN ei.start_at <= NOW() AND COALESCE(ei.end_at, ei.start_at) >= NOW() THEN TRUE ELSE
FALSE END AS is_happening_now,
6371 * acos(
cos(radians(:user_lat)) * cos(radians(v.latitude)) *
cos(radians(v.longitude) - radians(:user_lng)) +
sin(radians(:user_lat)) * sin(radians(v.latitude))
) AS distance_km
FROM event_instances ei
JOIN events e ON e.id = ei.event_id
JOIN venues v ON v.id = ei.venue_id
JOIN event_categories ec ON ec.event_id = e.id AND ec.is_primary = TRUE
JOIN categories pc ON pc.id = ec.category_id
WHERE e.status = 'published'
AND ei.status = 'scheduled'
AND v.latitude IS NOT NULL
AND v.longitude IS NOT NULL
AND COALESCE(ei.end_at, ei.start_at) >= NOW()
)
SELECT *
FROM preview_base
WHERE distance_km <= :radius_km
ORDER BY distance_km ASC, start_at ASC
LIMIT 50;

```

9.3 City page homepage modules query shape

Illustrative Query - Homepage and city feed query shapes

-- Example module: This Weekend in Mumbai

```
SELECT e.id, e.title, e.slug, v.name AS venue_name, ei.start_at
FROM event_instances ei
JOIN events e ON e.id = ei.event_id
JOIN venues v ON v.id = ei.venue_id
JOIN cities c ON c.id = e.city_id
WHERE c.slug = :city_slug
AND e.status = 'published'
AND ei.status = 'scheduled'
AND ei.start_at >= :weekend_start
AND ei.start_at < :weekend_end
ORDER BY ei.start_at ASC
LIMIT 30;
```

10. Structured Data for Google

Every event occurrence page should emit schema.org Event JSON-LD. This is critical for rich search visibility and for making event details understandable to search engines. Structured data should be emitted from the canonical occurrence page, not just the generic parent event shell.

Illustrative Payload - schema.org Event JSON-LD

```
{
  "@context": "https://schema.org",
  "@type": "Event",
  "name": "Open Mic Comedy Night",
  "startDate": "2026-04-10T20:30:00+05:30",
  "endDate": "2026-04-10T22:30:00+05:30",
  "eventAttendanceMode": "https://schema.org/OfflineEventAttendanceMode",
  "eventStatus": "https://schema.org/EventScheduled",
  "image": [
    "https://cdn.where.at.today/events/open-mic-hero.jpg"
  ],
  "description": "A weekly open mic featuring upcoming comics and guest headliners.",
  "location": {
    "@type": "Place",
    "name": "The Habitat",
    "address": {
      "@type": "PostalAddress",
      "streetAddress": "3rd Floor, Unicontinental, Khar West",
      "addressLocality": "Mumbai",
      "addressRegion": "Maharashtra",

```

```

    "addressCountry": "IN"
  }
},
"offers": {
  "@type": "Offer",
  "url": "https://seller.example/open-mic-apr-10",
  "price": "299.00",
  "priceCurrency": "INR",
  "availability": "https://schema.org/InStock"
},
"organizer": {
  "@type": "Organization",
  "name": "Comedy Club India"
}
}

```

Category icons, map marker keys, and venue amenity icons are presentation metadata and do not belong in schema.org Event JSON-LD.

11. URL Structure and SEO Rules

- Use clean, stable slugs from day one.
- Do not put raw numeric IDs or UUIDs into public URLs.
- Keep page patterns predictable so internal linking is easy.
- Create crawlable page types instead of relying only on client-side filters.
- Use one canonical URL per real-world occurrence.

```

/
/mumbai
/mumbai/comedy
/mumbai/music
/mumbai/today
/mumbai/tonight
/mumbai/weekend
/venue/the-habitat-khar
/organizer/comedy-club-india
/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030

```

Earlier revisions used a date-only occurrence idea in some places. V3.7 resolves the ambiguity by recommending a time token in the public route, treating canonical occurrence URLs as event-instance-owned rather than event-owned, and using `event_instances.canonical_url` as the strict public lookup key.

11.1 Slug rules

- lowercase only
- hyphen-separated words
- no special characters
- once published, avoid changing unless absolutely necessary
- if a slug changes, create a 301 redirect from the old URL to the new URL

11.2 Canonical occurrence routes and event-shell rules

The canonical public SEO page should always be the occurrence-specific page, not the generic event shell. Recommended rule: `/event/{event-slug}/{yyyy-mm-dd-hhmm}` is the canonical public route whenever the product exposes event pages outside city/category route trees. Canonical occurrence URL ownership belongs to event_instances, while events keep only the reusable shell slug. The event-shell endpoint such as `GET /api/events/{eventSlug}` is useful for discovery and internal data retrieval, but it should not be treated as the canonical SEO page renderer.

If the same event has two occurrences on the same date, the time token prevents ambiguity. This also makes redirects, structured data generation, analytics attribution, and reminder behavior cleaner.

11.2.1 Public route resolution for SSR and event-page fetches

The public canonical route should resolve through the exact `event_instances.canonical_url` value for the incoming request path. The human-readable route still contains the event shell slug plus occurrence token, but SSR lookup should use the canonical path as the strict public key. Zero matches should return not found, and more than one match would indicate a data-quality error that must fail loudly rather than silently choosing one occurrence.

Illustrative Query - Public route resolution

```
SELECT ei.id, ei.canonical_url, ei.start_at
FROM event_instances ei
WHERE ei.canonical_url = :request_path;
```

If this lookup returns zero rows, respond with not found. If bad data ever produces more than one matching row, treat that as a data-quality error and fail loudly rather than silently choosing one occurrence.

11.3 Slug governance, regeneration, and redirects

Slug governance should be intentionally strict so content operations and engineering never make conflicting assumptions.

- Published slugs should be considered stable identifiers, not editable cosmetics.
- Title changes do not automatically require slug changes.
- Venue changes do not automatically rewrite historical slugs even if the old slug contains venue context; slug changes should be an explicit editorial decision with redirect creation.
- If a slug must change, create a permanent 301 redirect from the old route to the new canonical route and log the reason in the audit trail.

- Slug regeneration should happen before publish or through a controlled admin action, not silently during background updates.

11.4 Lifecycle policy for cancellations, sold-out states, and past events

The guide should define how pages behave after the event state changes so users and search engines see consistent behavior.

- Cancelled occurrences should stay live on their original occurrence page with a clear cancelled banner, updated structured data status, and alternative recommendations where helpful.
- Sold-out events should remain discoverable because they still signal market demand and may lead users to future occurrences.
- Completed occurrences may remain accessible on their occurrence-specific pages for SEO and user trust, but the product can eventually decide whether older past events stay indexable or move to noindex archive handling after a chosen age threshold.
- Canonicalization should remain occurrence-specific; do not canonicalize past occurrences to the next or previous instance because that weakens history and confuses structured data.

12. Suggested API Surface

The API should reflect the product's main browsing modes - city-first browsing, category-first browsing within a city, time-first browsing such as today or weekend, event occurrence pages, and map-based browsing.

Endpoint	Purpose	Example
GET /api/cities	Return supported cities	/api/cities
GET /api/cities/{citySlug}/home	Return homepage modules for a city	/api/cities/mumbai/home
GET /api/cities/{citySlug}/events	Return paginated city event feed	/api/cities/mumbai/events?time=tonight
GET /api/cities/{citySlug}/categories/{categorySlug}	Return city + category feed	/api/cities/mumbai/categories/comedy
GET /api/cities/{citySlug}/time/{timeFilter}	Return a time-first feed	/api/cities/mumbai/time/weekend
GET /api/events/{eventSlug}	Return event shell + active instances	/api/events/open-mic-comedy-night-habitat-mumbai
GET /api/event-instances/{instanceId}	Return one exact occurrence payload	/api/event-instances/6666...
GET /api/map/events	Return map-ready nearby or city payload	/api/map/events?city=mumbai&time=weekend&category=comedy
GET /api/venues/{venueSlug}	Return venue page data	/api/venues/the-habitat-khar

POST /api/subscriptions	Create or update a topic follow or alert preference	/api/subscriptions
POST /api/newsletter/subscribe	Create a newsletter-only subscriber identity	/api/newsletter/subscribe

12.1 API response design principle

- Homepage modules should return grouped content blocks, not raw rows that force frontend joins.
- Event pages should return one rich occurrence payload.
- Map endpoints should return lightweight marker and preview payloads.
- The API should be shaped around page needs, not table boundaries.

12.2 Example response - city homepage

Illustrative Payload - Homepage API response

```
{
  "city": { "name": "Mumbai", "slug": "mumbai" },
  "modules": {
    "happeningNow": [],
    "tonight": [
      {
        "eventInstanceId": "666666666-6666-6666-6666-666666666666",
        "title": "Open Mic Comedy Night",
        "slug": "open-mic-comedy-night-habitat-mumbai",
        "startAt": "2026-04-10T20:30:00+05:30",
        "venueName": "The Habitat",
        "primaryCategory": { "slug": "comedy", "iconKey": "mic", "mapMarkerKey": "marker-comedy",
        "colorHex": "#FF6B6B" }
      }
    ]
  }
}
```

12.3 Example response - category feed

Illustrative Payload - Category feed API response

```
{
  "city": "mumbai",
  "category": "comedy",
  "items": [
    {
      "eventInstanceId": "666666666-6666-6666-6666-666666666666",
      "title": "Open Mic Comedy Night",
      "slug": "open-mic-comedy-night-habitat-mumbai",
      "startAt": "2026-04-10T20:30:00+05:30",

```

```

    "canonicalUrl": "/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030",
    "venueName": "The Habitat",
    "tags": ["18-plus"]
  }
]
}

```

12.4 Example response - event occurrence page

Illustrative Payload - Event occurrence API response

```

{
  "id": "55555555-5555-5555-5555-555555555555",
  "slug": "open-mic-comedy-night-habitat-mumbai",
  "title": "Open Mic Comedy Night",
  "instance": {
    "id": "66666666-6666-6666-6666-666666666666",
    "canonicalUrl": "/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030",
    "startAt": "2026-04-10T20:30:00+05:30",
    "endAt": "2026-04-10T22:30:00+05:30",
    "inventoryStatus": "available"
  },
  "primaryCategory": {
    "slug": "comedy",
    "iconKey": "mic",
    "mapMarkerKey": "marker-comedy",
    "colorHex": "#FF6B6B"
  },
  "venue": {
    "name": "The Habitat",
    "slug": "the-habitat-khar",
    "services": [
      { "name": "Parking Available", "iconKey": "parking", "highlighted": true },
      { "name": "Air Conditioned", "iconKey": "ac", "highlighted": true }
    ]
  },
  "offers": [
    { "sellerName": "BookMyShow", "priceMin": 299.0, "priceMax": 499.0, "currency": "INR" }
  ]
}

```

12.5 Required preview payload for /api/map/events

```
{
  "eventInstanceId": "66666666-6666-6666-6666-666666666666",
  "title": "Open Mic Comedy Night",
  "slug": "open-mic-comedy-night-habitat-mumbai",
  "canonicalUrl": "/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030",
  "startAt": "2026-04-10T20:30:00+05:30",
  "venue": {
    "name": "The Habitat",
    "latitude": 19.069600,
    "longitude": 72.836500
  },
  "primaryCategory": {
    "iconKey": "mic",
    "mapMarkerKey": "marker-comedy",
    "colorHex": "#FF6B6B"
  }
}
```

12.6 Optional enrichment fields and full example response

```
{
  "eventInstanceId": "66666666-6666-6666-6666-666666666666",
  "title": "Open Mic Comedy Night",
  "slug": "open-mic-comedy-night-habitat-mumbai",
  "canonicalUrl": "/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030",
  "startAt": "2026-04-10T20:30:00+05:30",
  "priceMin": 299.00,
  "inventoryStatus": "available",
  "isEditorPick": true,
  "thumbnailUrl": "https://cdn.where.at.today/events/open-mic-hero.jpg",
  "distanceKm": 4.2,
  "isHappeningNow": false,
  "venue": {
    "name": "The Habitat",
    "slug": "the-habitat-khar",
    "latitude": 19.069600,
    "longitude": 72.836500,
    "services": [
      {"name": "Parking", "iconKey": "parking"},
      {"name": "Air Conditioning", "iconKey": "ac"}
    ]
  },
  "primaryCategory": {
    "name": "Comedy",
    "slug": "comedy",
    "iconKey": "mic",
    "mapMarkerKey": "marker-comedy",
    "colorHex": "#FF6B6B"
  }
}
```

Suggested field set for strong map previews: priceMin, inventoryStatus, isEditorPick, thumbnailUrl, distanceKm, and isHappeningNow. thumbnailUrl should resolve from approved media only: first prefer the approved hero image, then fall back to the first approved non-hero asset by sort_order, and if none exists return null so the product can apply a defined fallback image policy. Highlighted venue services may be joined into preview responses when the product wants richer popovers without a second fetch.

12.7 Preview versus detail contract

Required preview fields for /api/map/events should include: eventInstanceId, title, slug, canonicalUrl, startAt, venue name, venue coordinates, primaryCategory iconKey, primaryCategory mapMarkerKey, and primaryCategory colorHex.

Optional enrichment fields for map previews include: priceMin, inventoryStatus, isEditorPick, thumbnailUrl, distanceKm, isHappeningNow, and highlighted venue services.

Detail-page-only fields such as fullDescription, media gallery, complete offers, organizer details, and long-form tags should remain on the event occurrence payload rather than being required on every map response.

Illustrative Payload - Map endpoint API response

```
[
  {
    "eventInstanceId": "66666666-6666-6666-6666-666666666666",
    "title": "Open Mic Comedy Night",
    "slug": "open-mic-comedy-night-habitat-mumbai",
    "canonicalUrl": "/event/open-mic-comedy-night-habitat-mumbai/2026-04-10-2030",
    "startAt": "2026-04-10T20:30:00+05:30",
    "priceMin": 299.0,
    "inventoryStatus": "available",
    "isEditorPick": true,
    "thumbnailUrl": "https://cdn.where.at.today/events/open-mic-hero.jpg",
    "distanceKm": 4.2,
    "isHappeningNow": false,
    "venue": {
      "name": "The Habitat",
      "slug": "the-habitat-khar",
      "latitude": 19.0696,
      "longitude": 72.8365,
      "services": [
        { "name": "Parking Available", "iconKey": "parking", "highlighted": true },
        { "name": "Air Conditioned", "iconKey": "ac", "highlighted": true }
      ]
    },
    "primaryCategory": {
      "name": "Comedy",
      "slug": "comedy",
      "iconKey": "mic",
```

```
"mapMarkerKey": "marker-comedy",  
"colorHex": "#FF6B6B"  
}  
}  
]
```

13. Internal Tools for the Curation Team

Manual curation will be the main quality engine at launch. Because of that, internal tools matter almost as much as the public product.

Event Entry Form

A structured event entry form should keep typing minimal. It should support title, category, primary category, city, venue search, date, time, ticket URL, source URL, image URL, tags, and status. The target should be about 30 seconds per event once the venue already exists.

Venue Manager

Curators should manage venues separately so event entry becomes a fast select-and-fill workflow instead of repeated typing. Venue services should also be editable here.

Organizer Manager

Useful for grouping promoter-led events and future organizer pages.

Duplicate Detection

The system should warn if the same or similar title exists for the same venue and date.

Venue Tracker

A simple table that shows Venue | City | Events Added | Last Checked will make weekly coverage disciplined.

Bulk Status Manager

Past, cancelled, and postponed events should be easy to update in batches.

Calendar View

This helps the team spot gaps and event density across a week or month.

Data Quality Warnings

The admin should block publish unless the record passes the canonical publish-ready checklist in Section 18.2: title, valid date/time, venue, valid venue coordinates, at least one category, exactly one primary category, canonical occurrence URL, one approved hero image, valid source or ticket data, no unresolved duplicate warning, and enough structured-data fields to emit a clean schema.org Event payload.

Event Search

Curators should be able to search quickly by title, venue, city, category, and date to check whether an event already exists or to edit incorrect information.

Image Preview and Broken Image Warning

The system should preview the chosen image before save and warn if the image URL is broken or returns an invalid file.

13.1 Maker-checker workflow and approval controls

Maker-checker should be universal across all human-managed data so the platform stays consistent. That includes events, event_instances, venues, organizers, categories, tags, event_media, event_ticket_offers, venue services, and other master data maintained through the curation admin.

Suggested roles: Maker creates or edits records, sees draft and changes-requested queues, and submits work for review. Checker reviews submitted records, compares the proposed version against the current approved version, approves, rejects, or requests changes. Admin manages roles, emergency overrides, master-data ownership, and audit visibility.

Suggested universal states should be read in three separate layers. Review workflow state: draft, pending_review, changes_requested, approved, rejected. Publication state: draft, published, archived. Business lifecycle state for real-world occurrences stays separate, for example scheduled, postponed, cancelled, and completed on event_instances. Autosave should only persist draft review state, and only an explicit submit action should move work into pending_review.

When a checker rejects or requests changes, the entity should return to the maker queue with review notes, changed-field summary, and resubmit controls. The loop should be: draft -> pending_review -> changes_requested -> maker update -> pending_review, or pending_review -> rejected if the record should not proceed at all.

13.2 Universal role model, reject-resubmit loop, and queue behavior

Makers should see queue views such as My Drafts, Needs Changes, Submitted, and Published By Me. Checkers should see Pending Review, Approved Today, Sent Back, and Rejected. Admins should additionally see queue health, unresolved duplicates, stale drafts, and exception overrides.

A rejected or changes-requested record should never disappear into the system. It should visibly return to the maker with who reviewed it, when it was reviewed, what changed since the prior approved version, and what must be fixed before resubmission.

At launch, consistency matters more than micro-optimizing review scope. It is better for all operational entities to use one understandable maker-checker pattern than to create exceptions that confuse curators and reviewers.

Universal maker-checker is implemented in two layers. Inline review and publish fields on events, event_instances, event_media, and event_ticket_offers are the current effective state used by product behavior, admin screens, and publish gating. The shared entity_versions and entity_reviews tables are the history, queueing, diffing, and review-evidence layer for those entities and for other human-managed records such as venues, organizers, categories, tags, and venue services so the workflow promise remains universal without duplicating full review columns on every table.

13.3 Draft autosave and recovery

Data-entry forms should autosave drafts every few seconds and on field blur, then show Saving... and Saved just now status to the maker. Draft autosave should also support recovery after refresh, accidental tab close, or browser interruption.

The recommended storage pattern is hybrid: relational tables remain the source of truth for validated structured records, while JSONB draft snapshots preserve incomplete form state that may span event, instance, media, and offer edits before the record is ready for commit.

Autosave must not trigger publish or review. It only protects work in progress.

13.4 Reviewer diff-first compare view

Checker review should be diff-first, not record-first. The review screen should compare the current approved version against the newly submitted version side by side, with only changed fields highlighted by default.

Show a change summary bar at the top, for example: 8 fields changed, 2 critical changes, 1 media change, 1 ticket change. Critical changes should be surfaced first: title, slug, canonical URL, start or end time, venue, primary category, hero image, ticket URL, price, and status.

For text fields, show old value and new value with inline emphasis where possible. For media, show old hero image and new hero image. For taxonomy, show added and removed chips. For venue or time changes, show absolute date, time, timezone, venue name, and locality so the checker can approve or reject quickly.

13.5 Review data and compare support in practice

A strong review system usually needs three artifacts: a current structured record in the main tables, a version snapshot of the record as submitted, and a computed diff summary that the checker can read quickly. That is why `entity_versions` and `entity_reviews` are useful even when the source of truth stays relational. Per-table fields answer current effective state; `entity_versions` and `entity_reviews` preserve history, compare evidence, and review workflow context.

This design also makes rollback, auditability, and historical comparison easier later without forcing the whole platform into JSON-first storage.

A maker-checker workflow should sit on top of the curation admin as soon as more than one person can create or publish content. The maker creates or edits the record. The checker independently reviews it before it becomes live.

Maker responsibilities: create or edit event, venue, organizer, category assignment, primary category, venue services, image, ticket offer, or canonical URL details.

Checker responsibilities: verify duplicate warnings, required fields, primary category choice, canonical URL, date and time, venue mapping, ticket URL validity, image quality, and any highlighted services or editorial flags.

Suggested state model: keep review workflow state, publication state, and business lifecycle state separate. Use review workflow values such as `draft`, `pending_review`, `changes_requested`, `approved`, and `rejected` for approval flow; use publication values such as `draft`, `published`, and `archived` for live visibility; and use occurrence lifecycle values such as `scheduled`, `postponed`, `cancelled`, and `completed` for `event_instances`.

Critical published-field edits should trigger re-review instead of going live immediately. At minimum, title, slug, primary category, start or end time, venue, hero image, and ticket offer should require checker approval.

13.6 Universal launch review scope by entity

Use maker-checker for all human-managed entities at launch so the workflow stays consistent. The highest-risk public entities - events, event_instances, event_media, and event_ticket_offers - should additionally use direct publish gating and stricter reviewer SLAs because they most directly affect user trust, SEO, and conversion.

The shared entity_versions and entity_reviews workflow should also cover venues, organizers, categories, tags, venue_services, performers, and other master data. These support entities may use lighter review turnaround rules, but they should still move through the same maker-checker statuses so the admin experience remains predictable.

The checker view should show a clear diff of changed fields and a short review note so the team can understand what changed and why it was approved or sent back.

14. Launch Data Strategy - Venue-First Manual Ingestion

- Build a starting list of 20 to 30 strong venues in Mumbai.
- Open the venue event page, calendar page, or upcoming page.
- Extract only structured fields: title, date, start time, venue, city, category, description, ticket link, source URL, and image URL.
- During ingestion, the curator should assign at least one category and exactly one primary category, plus optional tags.
- If the venue already exists, venue services should be reviewed and updated only when needed, not re-entered for each event.
- Track which venues were checked and when.
- Repeat weekly so the city feels active and fresh.

This approach is enough to get to a few hundred events per month in one city without scraping while keeping data quality high.

15. Category, Tag, and Venue Service Strategy

Keep categories intentionally limited because they drive navigation, page generation, map identity, and internal linking. Keep tags more flexible because they drive filtering, discovery enrichment, and editorial nuance. Venue services answer what the place is like.

15.1 Recommended launch categories

- Comedy
- Music
- Theatre
- Workshops
- Networking

- Art
- Food and Drink
- Film
- Markets and Festivals
- Wellness

15.2 Recommended tag examples

- Free
- Outdoor
- Family Friendly
- 18+
- Pet Friendly
- Beginner Friendly
- Open Air
- Indoor
- Premium

15.3 Recommended visual category mapping

- Comedy -> mic icon
- Music -> music note icon
- Theatre -> stage icon
- Workshops -> tools or notebook icon
- Wellness -> lotus or wellness icon

15.4 Recommended venue services starter set

Group	Examples
Access	Wheelchair Accessible, Valet Parking, Parking Available, Metro Nearby
Comfort	Air Conditioned, Seating Available, Restrooms, Family Friendly
Food and beverage	Food Available, Bar Available, Alcohol Served
Format	Indoor Venue, Outdoor Venue, Standing, Seated
Policies	18+, Pets Allowed, Dress Code

16. Geo Discovery and Map Features

- The current schema supports Events Near Me, Within 5 km, Within 10 km, Happening Now Near Me, and This Weekend Near Me as long as the venue table has reliable latitude and longitude values.

- User shares exact location, or the product falls back to the selected city center.
- Each event_instance points to a venue.
- Each venue stores coordinates.
- The query computes distance between the user and the venue.
- Results are filtered by time window and radius.
- Map cards should render category marker identity, venue services, and inventory state without requiring a second fetch for normal browsing.

16.1 Clustering and rendering rules

- Use clustering when density becomes visually noisy, especially in central city areas.
- Cluster markers should show count, but once expanded or zoomed in, each event should revert to its own primary category marker.
- When multiple event instances exist at the same venue, frontend may stack or list them in the selected venue popover, but each still needs its own canonical event link.
- For MVP, raw lat/lng with a Haversine-style filter is enough. For scale, move to PostGIS.

16.2 Performance guidance and scaling notes

The launch indexes are a good start, but the platform is heavily filtered by city, time, status, and upcoming-state. Backend planning should anticipate richer composite indexes as traffic and content volume grow.

Recommended candidates for later evaluation: (city_id, status, start_at), (venue_id, start_at), and partial indexes for upcoming published instances. Move from Haversine to PostGIS once nearby search becomes a hot path or when the city footprint and event volume make radius queries noticeably expensive.

Good caching targets include city homepage modules, category feeds, time-window feeds, and venue pages. Revalidation should be triggered by instance status changes, ticket-offer changes, and hero-image changes.

17. Engagement Loops That Should Be Included Early

- Similar Events. At the bottom of each event page, show similar events in the same city, same category, and close in date.
- Recently Added. A homepage block showing newly published events helps the site feel alive and gives users a reason to browse again.
- Editor Picks. A manually curated block gives the brand taste and protects quality during the early phase.
- Subscriptions. Even a basic email follow feature will help turn one-time search visitors into repeat users.
- Follow Venue. Users should later be able to follow a venue, not just a category or city.
- Follow Category in City. A user may want Comedy in Mumbai or Music in Bangalore rather than all events globally.

17.1 Optional user accounts and profiles

The product should remain fully usable without login. Anonymous browsing is still the right default for SEO, sharing, and low-friction discovery.

At the same time, the platform should support optional accounts for users who want to save city preference, favorite events, follow categories, follow venues, and receive alerts.

Recommended profile capabilities: display name, avatar, home city, preferred language, saved radius for nearby discovery, favorite venues, favorite events for broad interest, saved event instances for actual attendance intent, and followed categories within a city.

The important design rule is that accounts should enhance discovery, not gate discovery. A logged-out visitor should still be able to browse, search, and land on canonical event pages without friction.

17.2 Future user system and retention scope

Even if accounts are introduced in Phase 2, the guide should reserve space for the broader user system so current schema and API choices do not box the product in later.

Recommended future capabilities: saved events, saved event instances, saved venues, recent views, alert preferences, reminder flows, add-to-calendar links, and city-aware follows such as Comedy in Mumbai rather than Comedy everywhere.

A good rule of thumb is to keep the public event graph anonymous-first while allowing user state to enrich ranking, reminders, and retention once a visitor chooses to create an account.

17.3 Newsletter-only subscription and later account upgrade

A website visitor should be able to subscribe to the newsletter without creating a full account. This is the right low-friction growth path for an anonymous-first product because it captures interest without blocking discovery behind signup.

If the same verified email later creates an account, the system should link the existing newsletter_subscribers record to the new users record rather than creating a second identity. This preserves consent history, city or category preferences, unsubscribe state, and future reminder eligibility. Existing subscription rows should be updated by backfilling user_id onto the same rows rather than creating duplicate rows for the same follow target.

18. Recommended Phasing

Universal maker-checker should be in place before multiple curators can change live data directly.

Newsletter-only subscription can launch earlier because it does not require a full account system.

Optional user accounts, saved state, reminders, and richer retention flows fit naturally into Phase 2 after anonymous browsing, SEO pages, map flows, and core curation workflows are stable.

Phase	What to build	What to avoid
Phase 1 - Launch	Homepage, one city page, event pages, category pages, time pages, venue pages, manual curation admin, category visuals, primary category logic, map	Complex automation, overbuilt personalization, deep recommendation logic, full visual search

	payload contract, canonical URL structure	
Phase 2 - Quality and retention	Subscriptions, improved venue coverage, organizer pages, performer pages, better duplicate checks, richer venue filters	Premature service splits
Phase 3 - Scale	Semi-automated ingestion, PostGIS, richer search, trending scores, better editorial tooling, visual search experiments	Ad hoc schema drift without governance

18.1 Visual search (future scope)

Visual search is not launch-critical, but the platform can support it later once the visual taxonomy and image quality are strong enough. That future layer might include image-first category browse, poster-led event tiles, and map-plus-image exploration modes. It should be treated as a scale feature, not a substitute for getting the category visual system and map contracts right at launch.

18.2 Definition of publish-ready

High-value enforcement examples worth locking at launch: published events must have exactly one primary category; published event instances should have a canonical occurrence URL; published event shells should have one approved hero image; and published event instances should not point to inactive venues or inactive cities. Tag each one as database-enforced, admin-validation-enforced, or reviewer-enforced so implementation ownership is explicit.

Admin warnings should be upgraded into one canonical publish-readiness gate so content operations, maker-checker review, and publish actions all apply the same quality bar every time.

A publish-ready occurrence should have: title, valid date/time, venue, valid venue coordinates, at least one category, exactly one primary category, canonical occurrence URL, one approved hero image, a valid source URL or ticket URL, no unresolved duplicate warning, and enough structured-data fields to emit a clean schema.org Event payload. This is the single checklist referenced by admin warnings, publish actions, and reviewer sign-off.

Optional quality boosters for higher confidence include verified image status, content_confidence above a chosen threshold, and last_verified_at within the team freshness window.

18.3 Publish-readiness enforcement layers

Database-enforced examples: foreign keys, unique slugs, TIMESTAMPTZ typing, event_instances.canonical_url uniqueness, and partial unique indexes that allow only one primary category per event and one approved hero image per event.

Admin-validation-enforced examples: valid venue coordinates, valid source or ticket URL, canonical URL generation, image aspect-ratio checks, thumbnail presence, and unresolved duplicate warnings.

Reviewer-enforced examples: title quality, correct primary category choice, venue accuracy when sources conflict, image appropriateness, and confidence that postponed or cancelled lifecycle states are reflected correctly on the public page.

Use the enforcement layer matrix below when deciding whether a rule belongs in the database, the admin workflow, or reviewer judgment.

Rule type	Primary enforcement layer	Examples
Structural integrity	Database-enforced	Foreign keys, unique slugs, unique <code>event_instances.canonical_url</code> , allowed status constraints, and partial uniqueness for one <u>primary category per event</u> .
Operational completeness	Admin-validation-enforced	Valid venue coordinates, canonical URL generation, image aspect-ratio checks, source or ticket URL validity, and unresolved duplicate warnings.
Editorial and trust judgment	Reviewer-enforced	Title quality, correct category choice, venue accuracy when sources conflict, image appropriateness, and confidence in postponed or cancelled states.

19. Final Build Checklist for the Team

- Lock the URL structure before launch.
- Use UUIDs for all core entities and slugs for public routes.
- Treat `event_instances` as the source of truth for time-based features.
- Store venue coordinates from day one.
- Store `category icon_key`, `map_marker_key`, and `color_hex` in structured data, then normalize API output to `iconKey`, `mapMarkerKey`, and `colorHex` consistently.
- Require exactly one primary category per event.
- Implement `schema.org Event JSON-LD` on event pages.
- Build the curation admin early, not as an afterthought.
- Add venue services and highlighted services to venue management before the first real content pass.
- Return map-ready payloads that already include marker metadata and highlighted services.
- Use one canonical URL per event occurrence, including a time token where needed.
- Introduce maker-checker approval before more than one curator can publish directly.
- Keep browsing anonymous by default, but model optional user accounts, profiles, favorites, saved occurrences, follows, and alert preferences for the next phase.
- Define explicit status vocabularies and keep them consistent across admin, API, and analytics.
- Require exactly one approved hero image per event, and validate image health before publish.

- Store source provenance, verification timestamps, and curator/checker attribution for manually ingested content.
- Store timestamps canonically in UTC and return API times in ISO 8601 with offset.
- Keep cancelled and sold-out occurrences on their own canonical pages instead of collapsing them into a generic event shell.
- Design map responses to include enough preview data that marker popovers usually do not need a second fetch.
- Make maker-checker universal for human-managed entities so the admin follows one review pattern.
- Support autosave drafts and recovery separately from submit-for-review actions.
- Keep relational records as source of truth, but use JSONB draft snapshots and before/after review payloads for recovery and compare views.
- Build checker review as a diff-first experience with critical changes surfaced first.
- Allow newsletter-only subscription without requiring signup, then link the same verified email to a user account later.

20. Source Notes

This guide consolidates the discussion into one implementation-ready handoff.